

Université Hassan II de Casablanca

Faculté des Sciences Ben M'Sik

Département de Mathématiques et Informatique

Mémoire de fin de module pour l'obtention de la Licence Sciences

Mathématiques et Informatique

Option : Développement informatique

Sujet :

Le projet consiste en la réalisation d'une application web e-commerce appelée E-Store, permettant aux utilisateurs de consulter un catalogue de produits, créer un compte, gérer un panier et passer des commandes. Il s'inscrit dans une approche full stack en combinant un backend développé avec Spring Boot et Spring Data JPA, un frontend avec Angular, ainsi que l'utilisation de MongoDB pour la gestion des données documentaires comme les avis. L'objectif est de concevoir une application structurée, modulaire et conforme aux principes d'architecture logicielle moderne.

Présenté et soutenu le .. Juin 2026 par :

M'LISSA HANIA

KESSOU ANA

Sous L'encadrement de :

- Pr.

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

Dédicace

Nous dédions ce travail, fruit de plusieurs semaines d'efforts, de collaboration et de persévérance :

À nos chers parents, qui ont toujours été une source inépuisable d'amour, de soutien et de motivation. Leur confiance en nous, leurs sacrifices et leurs encouragements constants ont été essentiels à notre réussite.

À nos familles, pour leur présence précieuse, leur appui moral et leurs conseils qui nous ont accompagnés tout au long de notre parcours académique.

À nos amis et camarades, pour leur soutien, leur esprit d'entraide et les moments partagés qui ont rendu cette expérience plus enrichissante et agréable.

À notre encadrant, pour son accompagnement, sa disponibilité et ses orientations pertinentes qui nous ont permis de mener ce projet dans les meilleures conditions.

Enfin, à toutes les personnes qui, de près ou de loin, ont contribué à l'aboutissement de ce travail et nous ont encouragés à donner le meilleur de nous-mêmes.

Remerciement

Dans le cadre de la réalisation de ce mini-projet, nous tenons à exprimer nos sincères remerciements à toutes les personnes qui ont contribué, de près ou de loin, à l'aboutissement de ce travail.

Nous adressons nos plus vifs remerciements à notre prof et encadrant, Omar Zahour, pour son encadrement, sa disponibilité, ses conseils avisés ainsi que pour la qualité de son accompagnement tout au long de ce module. Ses orientations et son expertise ont été d'une grande aide dans la compréhension des concepts et la réalisation de notre application.

Nous remercions également l'ensemble des enseignants de la Faculté des Sciences Ben M'Sick pour la formation de qualité qu'ils nous ont dispensée, ainsi que pour les connaissances théoriques et pratiques qui ont constitué une base solide pour la réalisation de ce projet.

Nous exprimons aussi notre gratitude envers nos familles et nos proches pour leur soutien moral, leur patience et leurs encouragements constants.

Enfin, nous remercions toutes les personnes qui ont contribué, directement ou indirectement, à la réussite de ce travail.

Table des matières

Dédicace	3
.....	3
Remerciement	4
Liste des figures	7
Liste des tableaux.....	8
Liste des diagrammes	9
Tableau des abréviations	10
Introduction générale	11
Chapitre 1 : cahier de charge	13
1.1 Introduction	14
1.2 Contexte du projet.....	14
1.3 Objectifs du projet	14
1.4 Acteurs du système	15
1.5 Besoins fonctionnels	15
1.6 Besoins non fonctionnels	16
1.7 Contraintes techniques	16
1.8 Planning prévisionnel.....	17
1.9 Conclusion du chapitre.....	17
CHAPITRE2: Analyse et Conception	18
Introduction	19
1. Phase 1 : Analyse.....	19
1.1 Recueil des besoins	19
1.2 Étude de l'existant	20
1.3 Cadrage du projet.....	20
1.4 Rédaction du cahier des charges.....	20

2. Phase 2 : Conception.....	21
2.1 Cas d'utilisation.....	21
2.2 Diagrammes UML	22
1. Diagramme de cas d'utilisation (Use Case Diagram).....	22
2. Diagramme de classes	24
3. Diagramme de séquence.....	27
2.3 Conception de la base de données	29
2.4 Maquettes	30
Conclusion du chapitre	30

Liste des figures

Liste des tableaux

Liste des diagrammes

Tableau des abréviations

Introduction générale

Avec l'essor des technologies numériques et le développement rapide du commerce électronique, les applications e-commerce sont devenues des outils indispensables dans la vie quotidienne des utilisateurs ainsi que dans le monde des affaires. Elles permettent de faciliter l'accès aux produits, d'optimiser les processus d'achat et d'offrir une expérience utilisateur fluide et interactive.

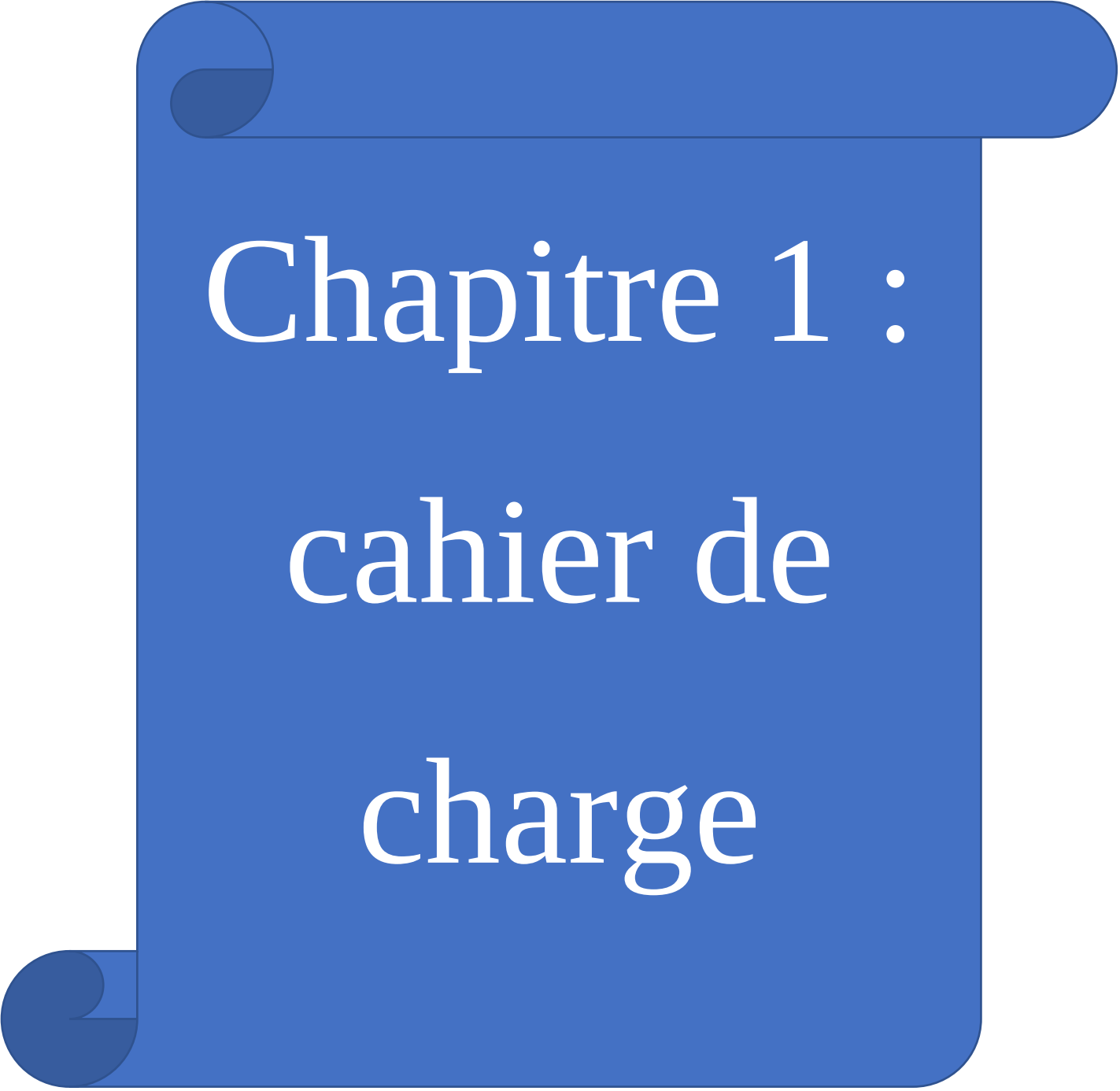
Dans ce contexte, le présent projet s'inscrit dans le cadre du module Full Stack et consiste en la conception et la réalisation d'une application web e-commerce intitulée E-Store. Cette application vise à reproduire le fonctionnement d'une boutique en ligne en intégrant les principales fonctionnalités telles que la gestion des utilisateurs, la consultation d'un catalogue de produits, la gestion du panier ainsi que la validation des commandes.

Au-delà de l'aspect fonctionnel, ce projet met l'accent sur l'adoption d'une architecture logicielle moderne et bien structurée. Il repose sur une approche en couches permettant de séparer clairement la présentation, la logique métier et l'accès aux données. Le backend est développé à l'aide du Framework Spring Boot avec l'utilisation de Spring Data JPA pour la gestion des données relationnelles, tandis que MongoDB est intégrée pour le traitement des données documentaires. Le frontend, quant à lui, est réalisé avec Angular afin d'assurer une interface dynamique et interactive.

Ce travail a pour objectif non seulement de mettre en pratique les connaissances acquises durant la formation, mais également de développer des compétences en conception logicielle, en structuration d'applications, en intégration de technologies multiples et en travail collaboratif. Il constitue ainsi une étape importante dans la transition d'un apprentissage théorique vers une approche professionnelle orientée ingénierie logicielle.

Dans ce rapport, nous présenterons dans un premier temps l'analyse des besoins et la spécification fonctionnelle du système, puis la conception détaillée à travers différents diagrammes UML. Ensuite, nous aborderons la phase de réalisation incluant le

développement backend et frontend, avant de conclure par une présentation des tests effectués et des perspectives d'amélioration.

A blue scroll graphic with a white border, featuring a rolled-up top edge and a rolled-up bottom-left corner. The text is centered on the scroll.

Chapitre 1 : cahier de charge

1.1 Introduction

Dans ce chapitre, nous présentons une étude détaillée du projet E-Store à travers une analyse des besoins fonctionnels et non fonctionnels. Cette étape constitue une phase essentielle dans le cycle de développement, car elle permet de comprendre les attentes des utilisateurs, de définir les fonctionnalités du système et d'identifier les contraintes à respecter.

L'objectif de cette analyse est de poser une base solide pour la conception et la réalisation de l'application, tout en garantissant une cohérence entre les besoins exprimés et les solutions techniques proposées.

1.2 Contexte du projet

Avec la croissance continue du commerce électronique, les plateformes de vente en ligne sont devenues indispensables pour faciliter les transactions et améliorer l'expérience utilisateur.

Dans ce cadre, le projet E-Store consiste à concevoir et développer une application web e-commerce permettant aux utilisateurs de consulter un catalogue de produits, gérer un panier et effectuer des commandes en ligne.

Ce projet s'inscrit dans une approche pédagogique visant à appliquer les concepts de développement full stack, en utilisant des technologies modernes telles que Spring Boot pour le backend, Angular pour le frontend, ainsi que JPA et MongoDB pour la gestion des données.

1.3 Objectifs du projet

Objectif général

Concevoir et développer une application e-commerce fonctionnelle, structurée et conforme aux principes d'architecture logicielle moderne.

Objectifs spécifiques

Mettre en œuvre une architecture en couches (Frontend / Backend / Data)

Développer des API REST avec Spring Boot

Gérer les données relationnelles avec JPA

Intégrer MongoDB pour les données documentaires

Concevoir une interface utilisateur dynamique avec Angular

Assurer la communication entre les différentes couches du système

1.4 Acteurs du système

Le système E-Store met en interaction plusieurs acteurs :

Client (Utilisateur) : utilisateur principal de l'application, il peut consulter les produits, gérer son panier et passer des commandes.

Administrateur : responsable de la gestion des produits, des catégories et du stock.

Système : assure les traitements automatiques tels que la vérification du stock, le calcul des totaux et l'enregistrement des commandes.

1.5 Besoins fonctionnels

Les besoins fonctionnels décrivent les fonctionnalités que le système doit offrir :

Gestion des comptes

- Inscription des utilisateurs
- Authentification (connexion)
- Gestion du profil

Gestion du catalogue

- Consultation des produits
- Recherche et filtrage
- Affichage des détails

Gestion du panier

- Ajout de produits
- Modification des quantités
- Suppression d'articles
- Calcul du total

Gestion des commandes

- Validation des commandes
- Enregistrement des commandes
- Consultation de l'historique

Gestion du stock

- Vérification de la disponibilité
- Mise à jour du stock

Gestion des avis

- Ajout d'avis
- Consultation des avis

1.6 Besoins non fonctionnels

Les besoins non fonctionnels concernent la qualité du système :

- **Performance** : temps de réponse rapide
- **Sécurité** : protection des données utilisateurs
- **Fiabilité** : gestion des erreurs et stabilité
- **Ergonomie** : interface simple et intuitive
- **Maintenabilité** : code structuré et lisible
- **Scalabilité** : la possibilité d'amélioration du plateforme

1.7 Contraintes techniques

Le projet doit respecter certaines contraintes techniques :

- Utilisation de Spring Boot pour le backend
- Utilisation de Angular pour le frontend
- Utilisation de JPA (Hibernate) pour la base relationnelle
- Intégration de MongoDB pour les données documentaires
- Architecture en couches obligatoire
- Communication via API REST
- Utilisation de Maven pour la gestion des dépendances

1.8 Planning prévisionnel

Le développement du projet est organisé selon un planning structuré :

Phases	Tâches principales
Phase 1	Analyse des besoins et conception
Phase 2	Développement Backend
Phase 3	Développement Frontend
Phase 4	Tests, intégration et finalisation

phases d'organisation 1

1.9 Conclusion du chapitre

Dans ce chapitre, nous avons présenté une analyse complète du projet E-Store en définissant le contexte, les objectifs, les acteurs ainsi que les besoins fonctionnels et non fonctionnels. Nous avons également identifié les contraintes techniques et établi un planning prévisionnel pour organiser le travail.

Cette analyse constitue une base essentielle pour la phase de conception, qui sera abordée dans le chapitre suivant à travers la modélisation du système à l'aide des diagrammes UML.

CHAPITRE2:

Analyse et Conception

Introduction

La phase d'analyse et de conception constitue une étape fondamentale dans le cycle de développement du projet. Elle permet de transformer les besoins exprimés dans le cahier des charges en une modélisation claire et structurée du système.

Dans le cadre du projet E-Store, cette phase vise à identifier les acteurs, définir les fonctionnalités principales, analyser l'existant et concevoir une architecture adaptée pour le développement d'une application e-commerce moderne et évolutive.

1. Phase 1 : Analyse

1.1 Recueil des besoins

Cette étape consiste à identifier les besoins des différents acteurs du système E-Store :

Client (Utilisateur)

- Consulter le catalogue de produits
- Rechercher et filtrer les produits
- Ajouter des produits au panier
- Passer des commandes
- Consulter l'historique des commandes
- Ajouter et consulter des avis

Administrateur

- Gérer les produits (ajout, modification, suppression)
- Gérer les catégories
- Gérer le stock
- Consulter les commandes
- Superviser les avis

Système

- Vérifier la disponibilité du stock
- Calculer le total du panier
- Enregistrer les commandes

- Gérer les erreurs

Méthodes utilisées :

- Analyse du cahier des charges
- Étude des plateformes e-commerce existantes
- Identification des besoins utilisateurs

1.2 Étude de l'existant

Avant la mise en place du système E-Store, les solutions existantes présentent plusieurs limites :

- Absence d'une plateforme centralisée pour la gestion des achats
- Difficulté de gestion des produits et du stock
- Manque d'automatisation dans le processus de commande
- Expérience utilisateur limitée et non optimisée

Cette situation justifie la conception d'une solution e-commerce intégrée permettant d'améliorer la gestion et l'expérience utilisateur.

1.3 Cadrage du projet

Le projet E-Store vise à développer une application web permettant :

- La gestion des utilisateurs (inscription, authentification, profil)
- La gestion du catalogue de produits
- La gestion du panier et des commandes
- La gestion du stock
- L'intégration d'un module d'avis (MongoDB)

Le système doit être structuré, modulaire et évolutif.

1.4 Rédaction du cahier des charges

Le cahier des charges définit :

- Les besoins fonctionnels du système

- Les exigences non fonctionnelles
- Les contraintes techniques
- Le planning de réalisation
- Les livrables attendus

Il constitue une référence essentielle pour toutes les étapes du projet.

2. Phase 2 : Conception

2.1 Cas d'utilisation

Les cas d'utilisation permettent de décrire les interactions entre les acteurs et le système.

Pour le client :

- S'inscrire et se connecter
- Consulter les produits
- Rechercher et filtrer les produits
- Ajouter au panier
- Modifier le panier
- Passer une commande
- Consulter les commandes
- Ajouter un avis

Pour l'administrateur :

- Gérer les produits
- Gérer les catégories
- Gérer le stock
- Consulter les commandes
- Gérer les avis

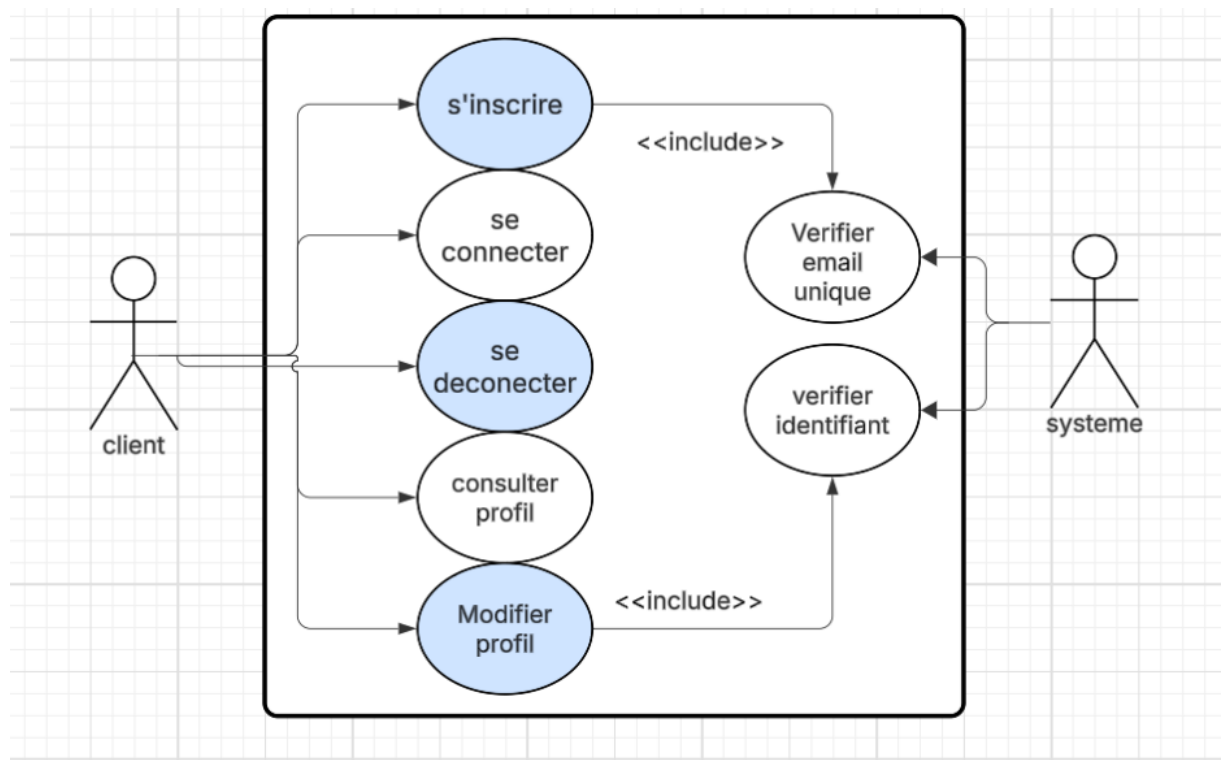
Pour le système :

- Vérifier le stock
- Calculer le total
- Enregistrer les commandes

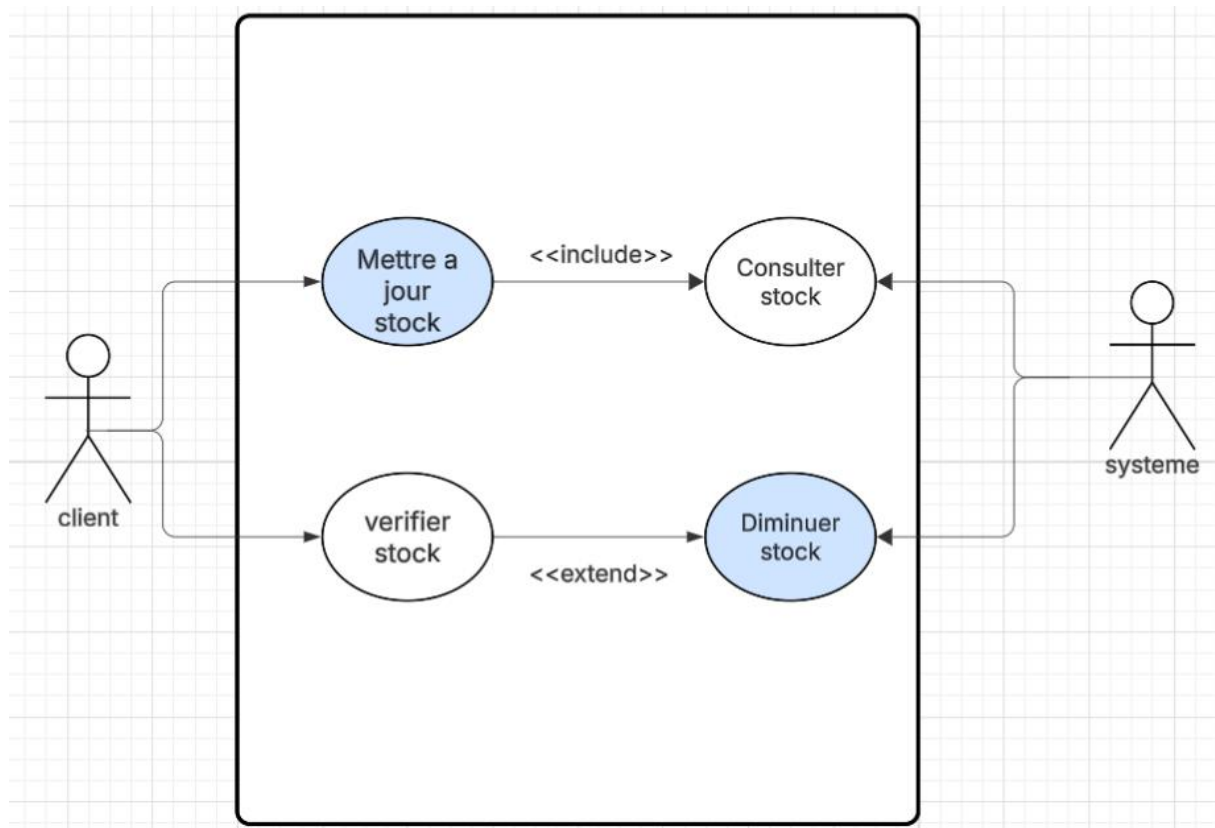
2.2 Diagrammes UML

1. Diagramme de cas d'utilisation (Use Case Diagram)

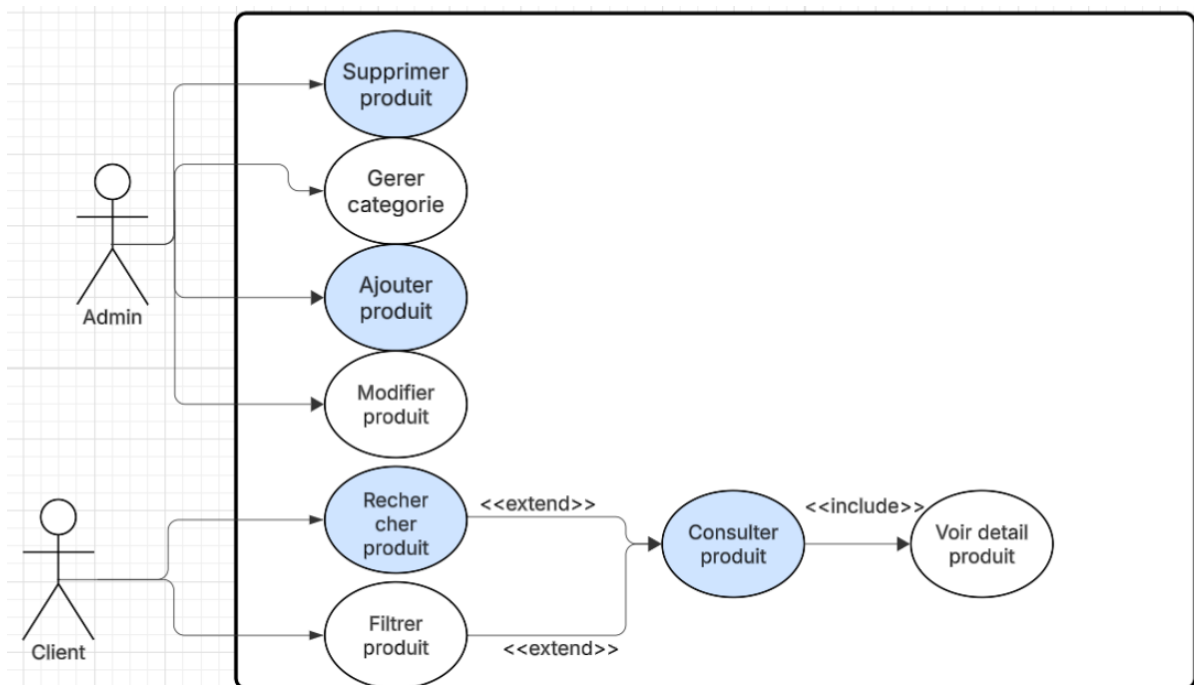
Ces diagrammes représentent les interactions entre les acteurs et le système E-Store. Il permet de visualiser les différentes fonctionnalités offertes ainsi que les rôles de chaque acteur.



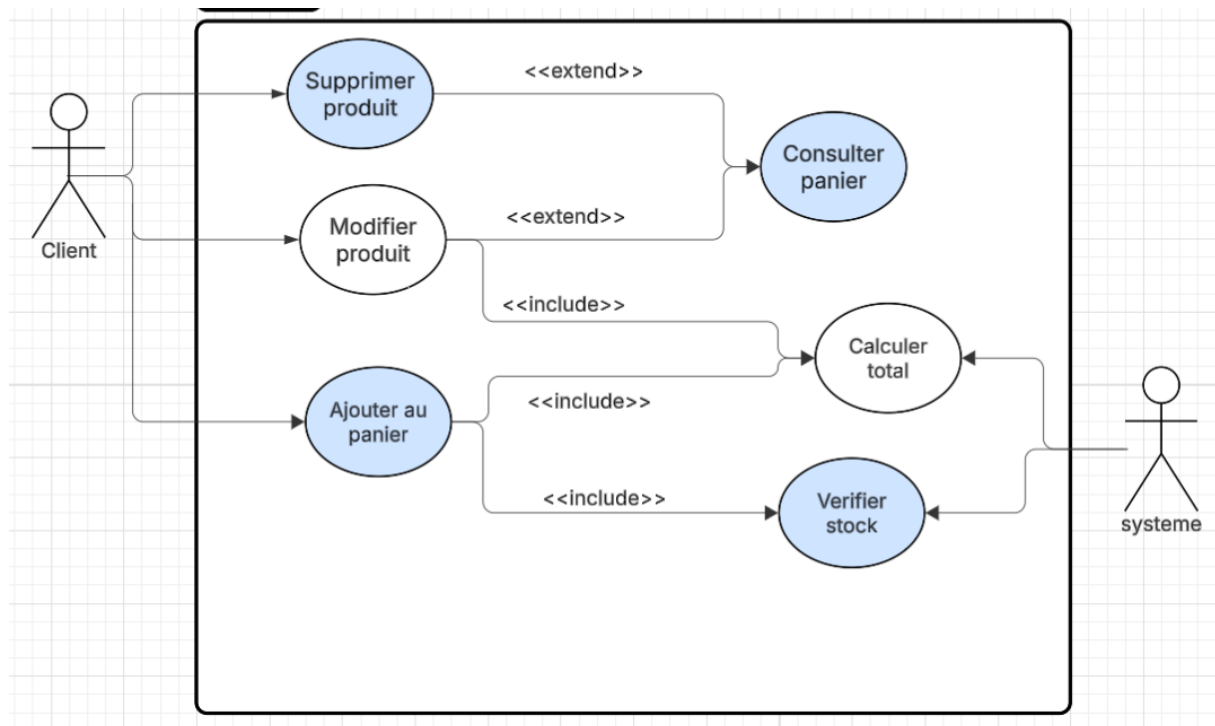
Use case : gestion des comptes clients



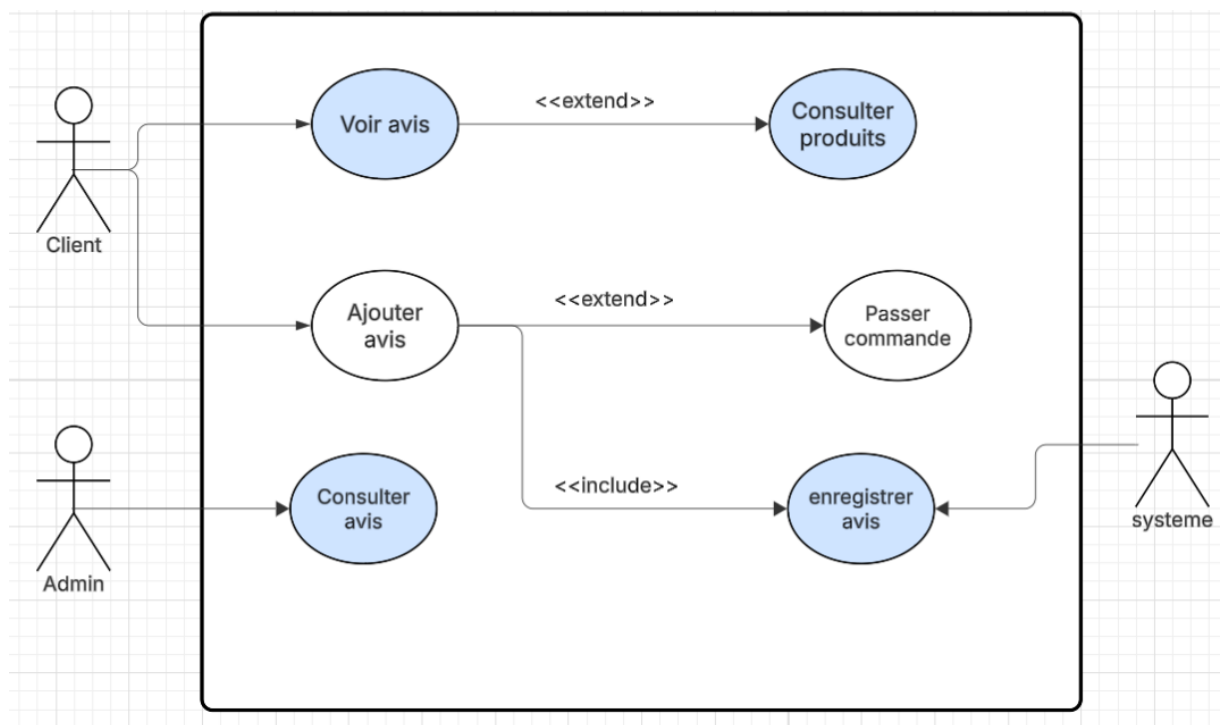
Use case : Gestion de stock (Inventory)



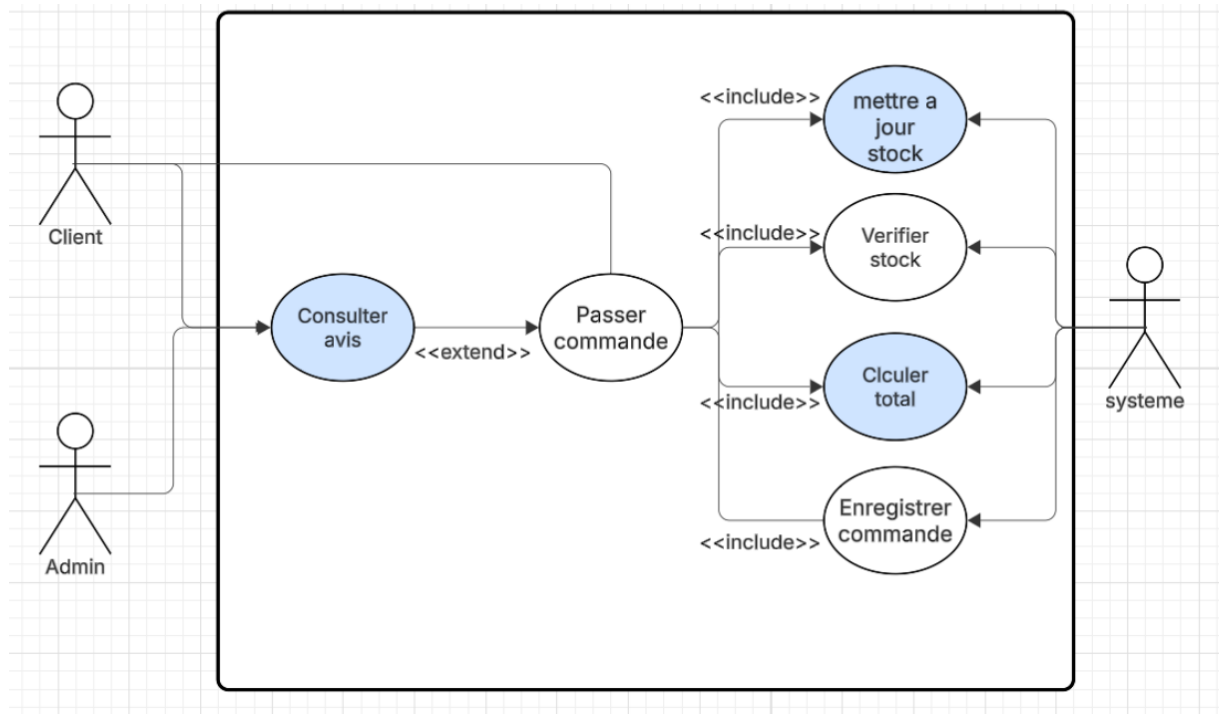
Use case : Gestion du catalogue



Use case : Gestion du panier



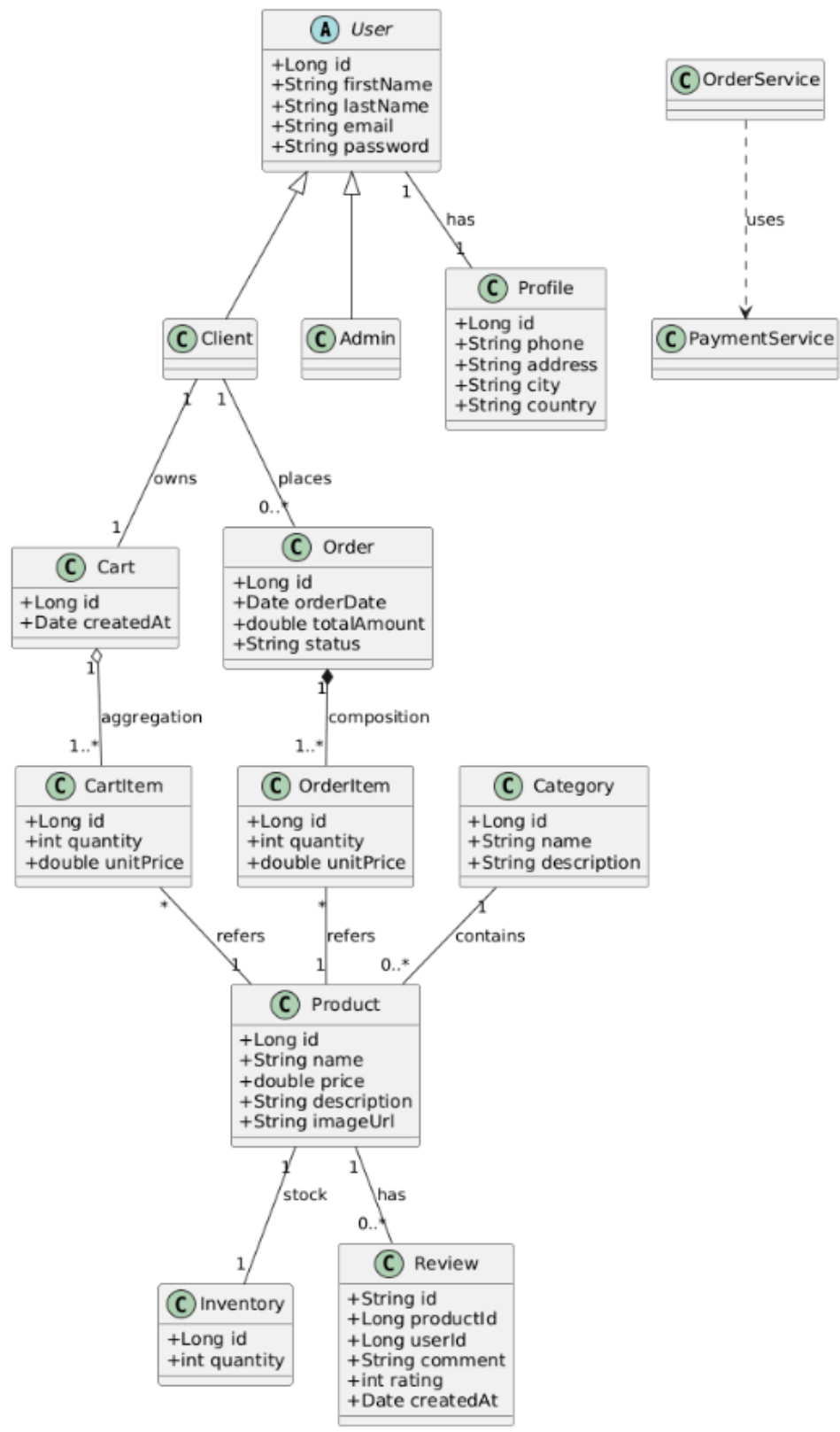
Use case : Gestion des avis (Mongo DB)



Use case : Gestion des commandes (Billings)

2. Diagramme de classes

Ce diagramme décrit la structure statique du système en présentant les différentes entités ainsi que leurs relations.



3. Diagramme de séquence

Ces diagrammes illustrent les interactions dynamiques entre les composants du système lors de l'exécution d'un scénario

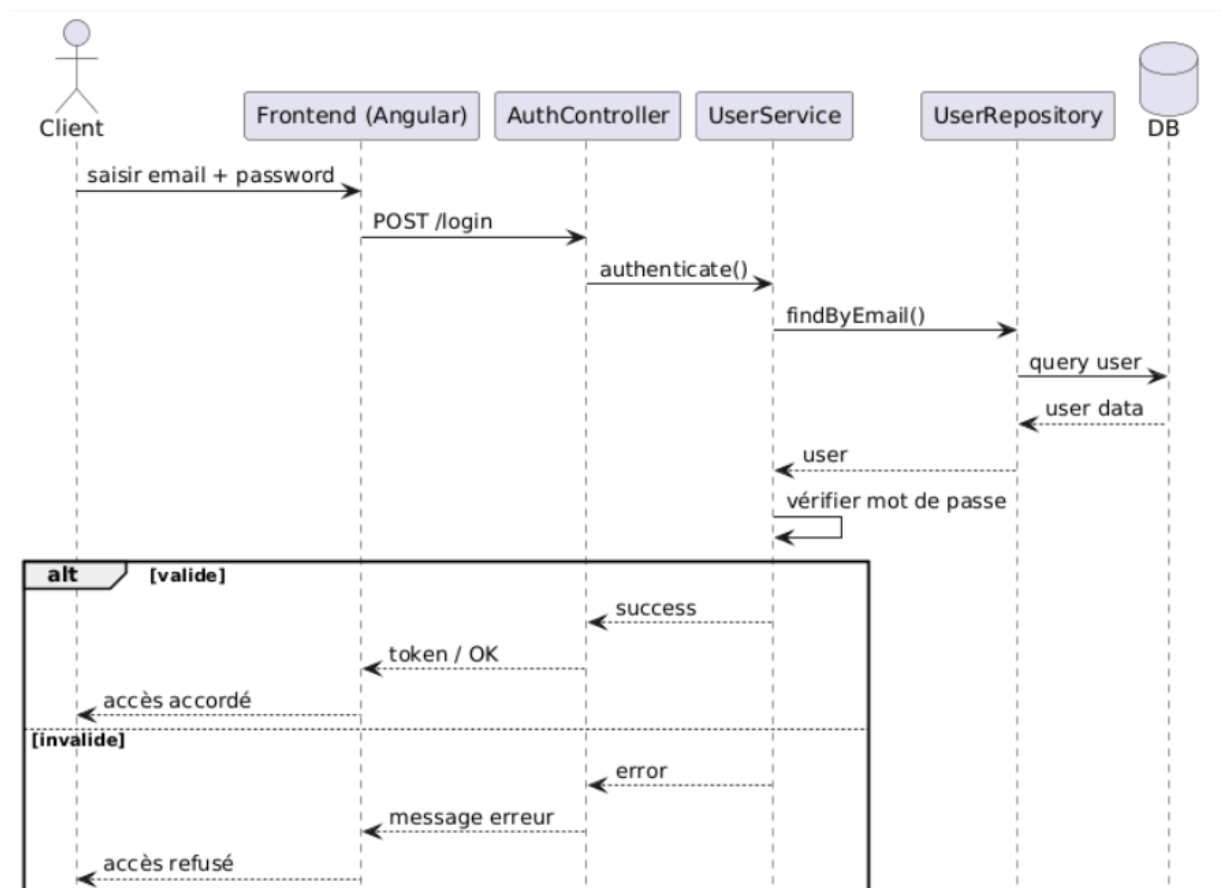


Diagramme de séquence : Authentification

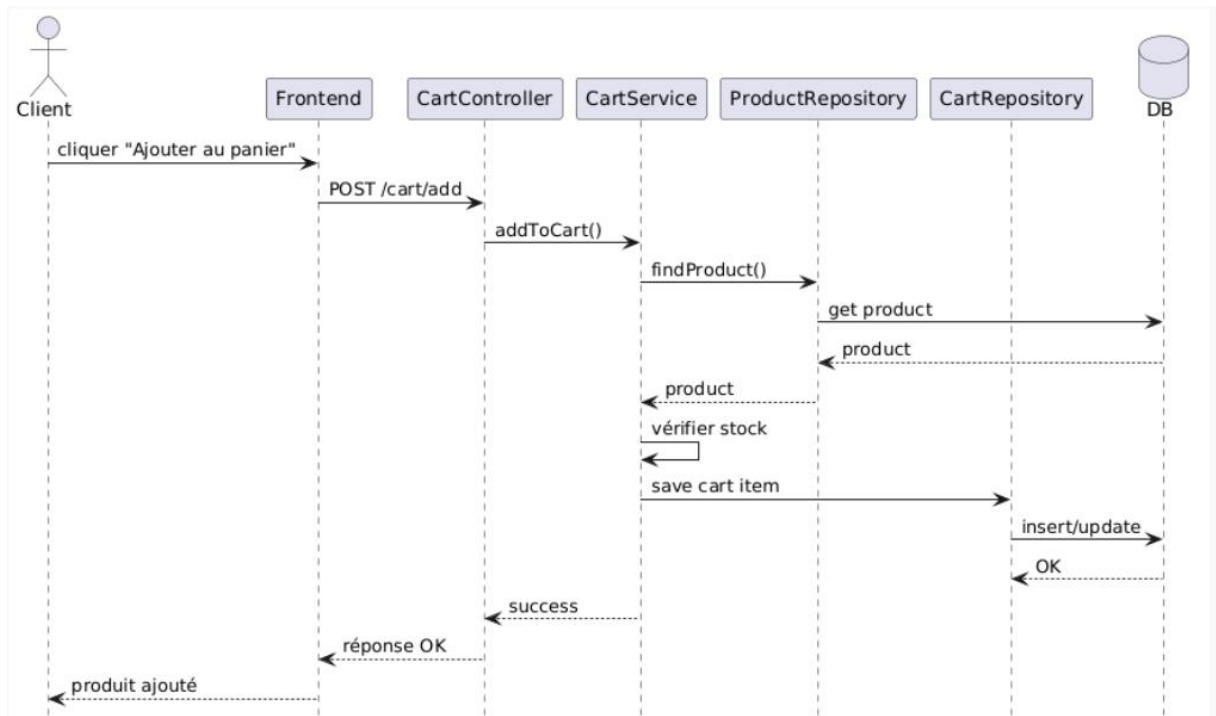


Diagramme de séquence : Ajouter au panier

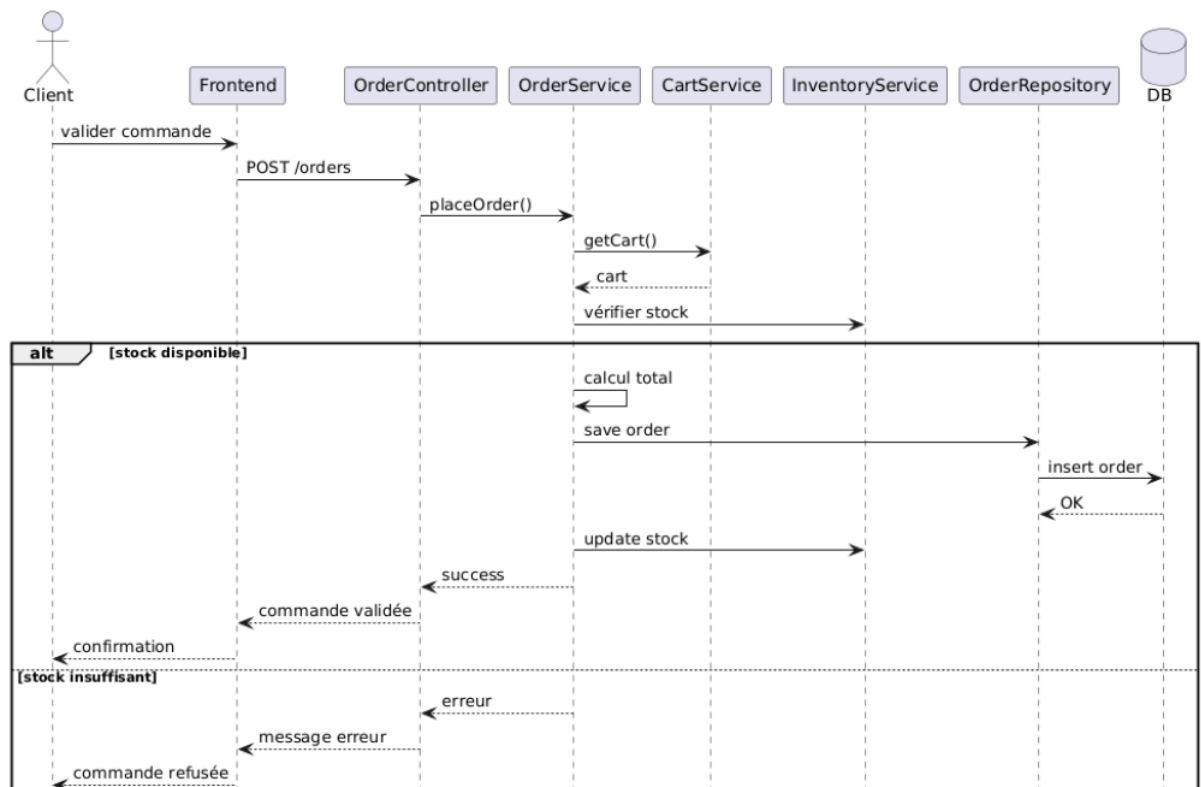


Diagramme de séquence : Passer commande

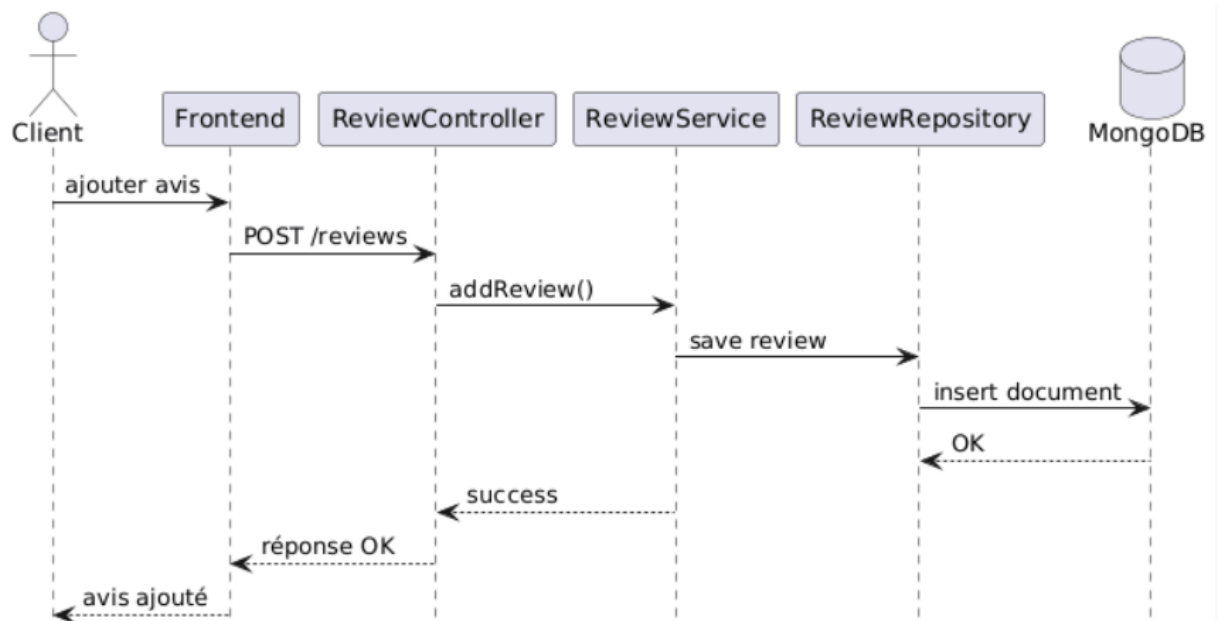


Diagramme de séquence : Ajouter un avis

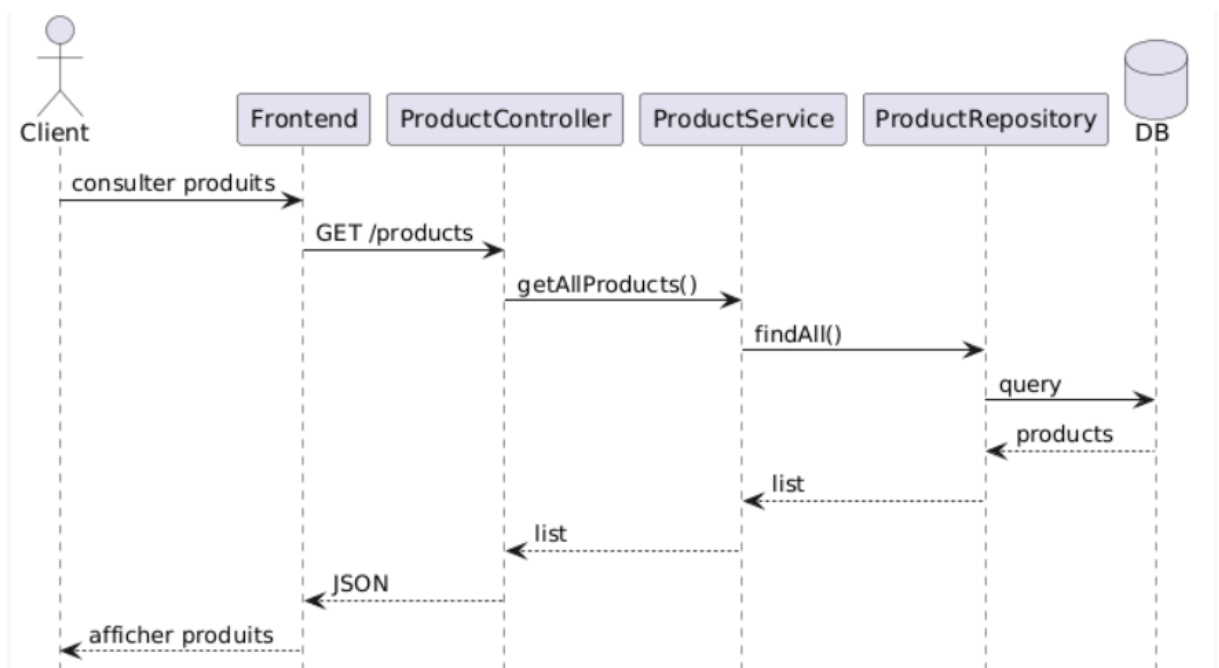


Diagramme de séquence : Consulter catalogue

2.3- Maquettes

Des maquettes ont été conçues pour représenter l'interface utilisateur, notamment :

- Page d'accueil
- Catalogue produits
- Détail produit
- Panier
- Page de commande
- Authentification

2.4 - Logo



Conclusion du chapitre

Dans ce chapitre, nous avons présenté une analyse détaillée du projet E-Store ainsi que la conception du système à travers les cas d'utilisation et les différents diagrammes UML.

Cette phase nous a permis de structurer le projet, de clarifier les interactions entre les acteurs et de définir une architecture cohérente. Elle constitue une base essentielle pour la phase de réalisation qui sera abordée dans le chapitre suivant

A blue scroll graphic with a white border, featuring a dark blue header bar and a dark blue footer bar. The main body of the scroll is a lighter blue color. The text is centered in the main body.

Chapitre 3 : Réalisation

1- Introduction

Dans ce chapitre, nous présentons la phase de réalisation du projet **E-Store**, qui consiste à implémenter les différentes fonctionnalités définies lors de la phase d'analyse et de conception.

Cette étape vise à concrétiser les modèles théoriques en une application fonctionnelle, en utilisant des technologies modernes du développement web, notamment Spring Boot pour le backend et Angular pour le frontend.

2- Environnement de développement

La réalisation du projet a été effectuée en utilisant un ensemble d'outils et de technologies adaptés au développement d'applications web modernes :

- **Langages de programmation** : Java, TypeScript, HTML, CSS
- **Framework Backend** : Spring Boot
- **Framework Frontend** : Angular
- **Base de données relationnelle** : MySQL (via JPA / Hibernate)
- **Base de données NoSQL** : MongoDB
- **Outils de développement** :
 - IntelliJ IDEA
 - Visual Studio Code
 - Postman
 - Git & GitHub

Technology Stack

- **Framework:** Spring Boot 3.2.0
- **Build Tool:** Maven
- **Database:** MySQL with Spring Data JPA
- **Document Database:** MongoDB for product reviews
- **Security:** Spring Security with JWT Authentication
- **API:** RESTful JSON APIs

Prerequisites

- Java 17+
- Maven 3.6+
- MySQL 8.0+
- MongoDB 6.0+

3- Architecture du système

```
src/main/java/com/estore/
├── EStoreApplication.java
├── billing/           # Order management
│   ├── controller/
│   ├── dto/
│   ├── entity/
│   ├── repository/
│   └── service/
├── catalog/          # Products and categories
│   ├── controller/
│   ├── dto/
│   ├── entity/
│   ├── repository/
│   └── service/
├── config/           # Configuration classes
├── customer/          # User authentication
│   ├── controller/
│   ├── dto/
│   ├── entity/
│   ├── repository/
│   └── service/
├── exception/         # Exception handling
├── inventory/         # Stock management
│   ├── controller/
│   ├── dto/
│   ├── entity/
│   ├── repository/
│   └── service/
├── reviews/          # Product reviews (MongoDB)
│   ├── controller/
│   ├── document/
│   ├── dto/
│   ├── repository/
│   └── service/
├── security/          # JWT security
├── shared/            # Common DTOs
└── shopping/          # Cart operations
    ├── controller/
    ├── dto/
    ├── entity/
    ├── repository/
    └── service/
```

L'architecture du système E-Store repose sur une approche en couches (Layered Architecture), permettant une séparation claire des responsabilités :

- **Frontend (Angular)** : interface utilisateur interactive
- **Backend (Spring Boot)** : gestion de la logique métier et des API REST
- **Base de données** : stockage des données (MySQL + MongoDB)

La communication entre le frontend et le backend se fait par API REST utilisant le format JSON.

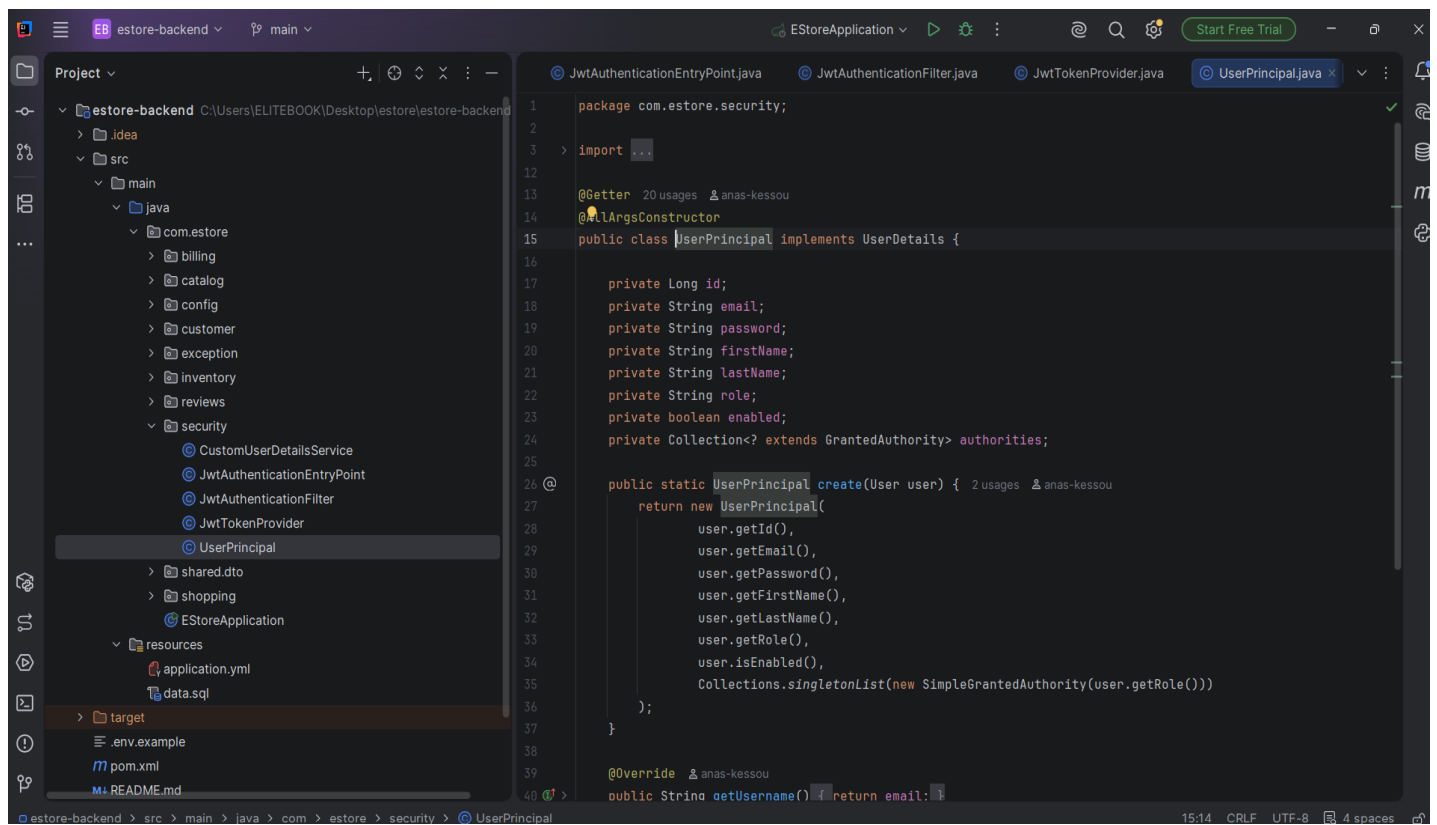
4- Réalisation du Backend

Le backend de l'application a été développé par Spring Boot en respectant une architecture en couches :

+ Couche Controller

Elle assure la gestion des requêtes HTTP et l'exposition des API REST.
Exemples :

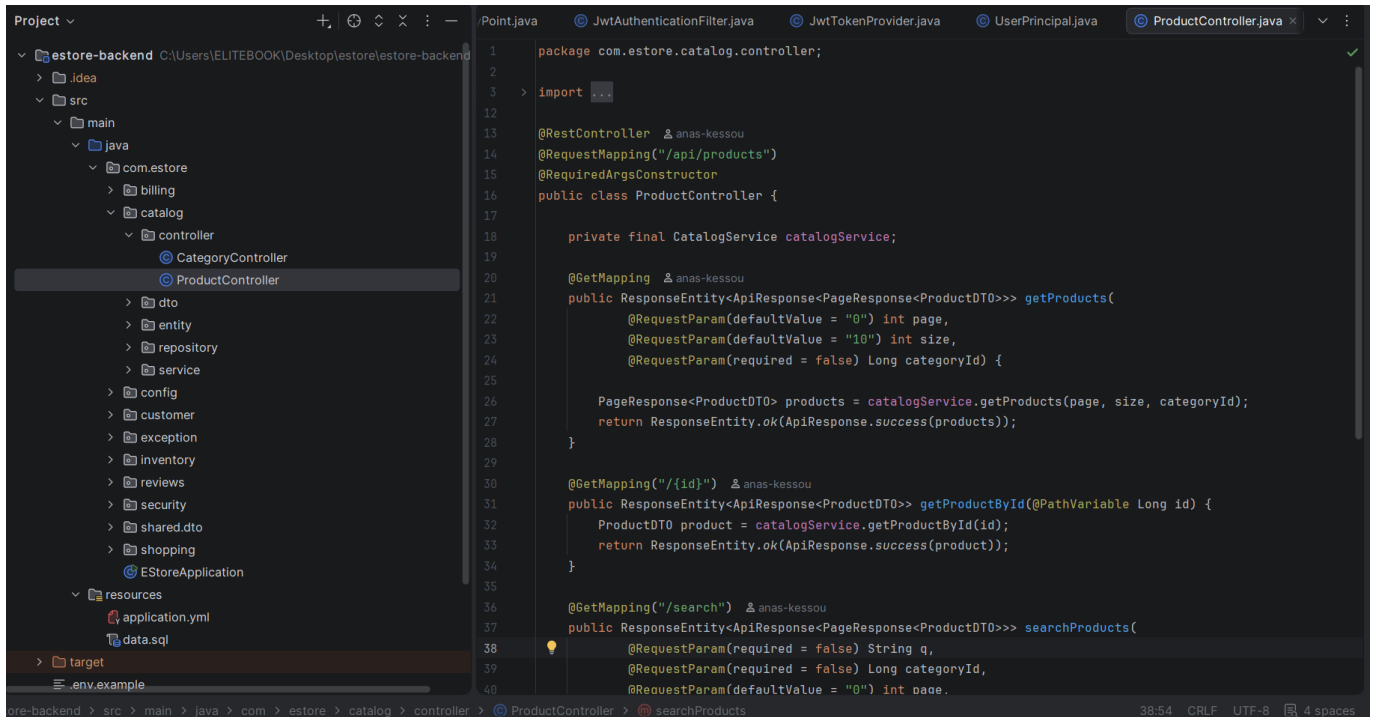
- AuthController



The screenshot shows an IDE with the project structure on the left and the code for `UserPrincipal.java` on the right. The project structure includes a `security` package with `CustomUserDetailsService`, `JwtAuthenticationEntryPoint`, `JwtAuthenticationFilter`, `JwtTokenProvider`, and `UserPrincipal`. The code for `UserPrincipal.java` is as follows:

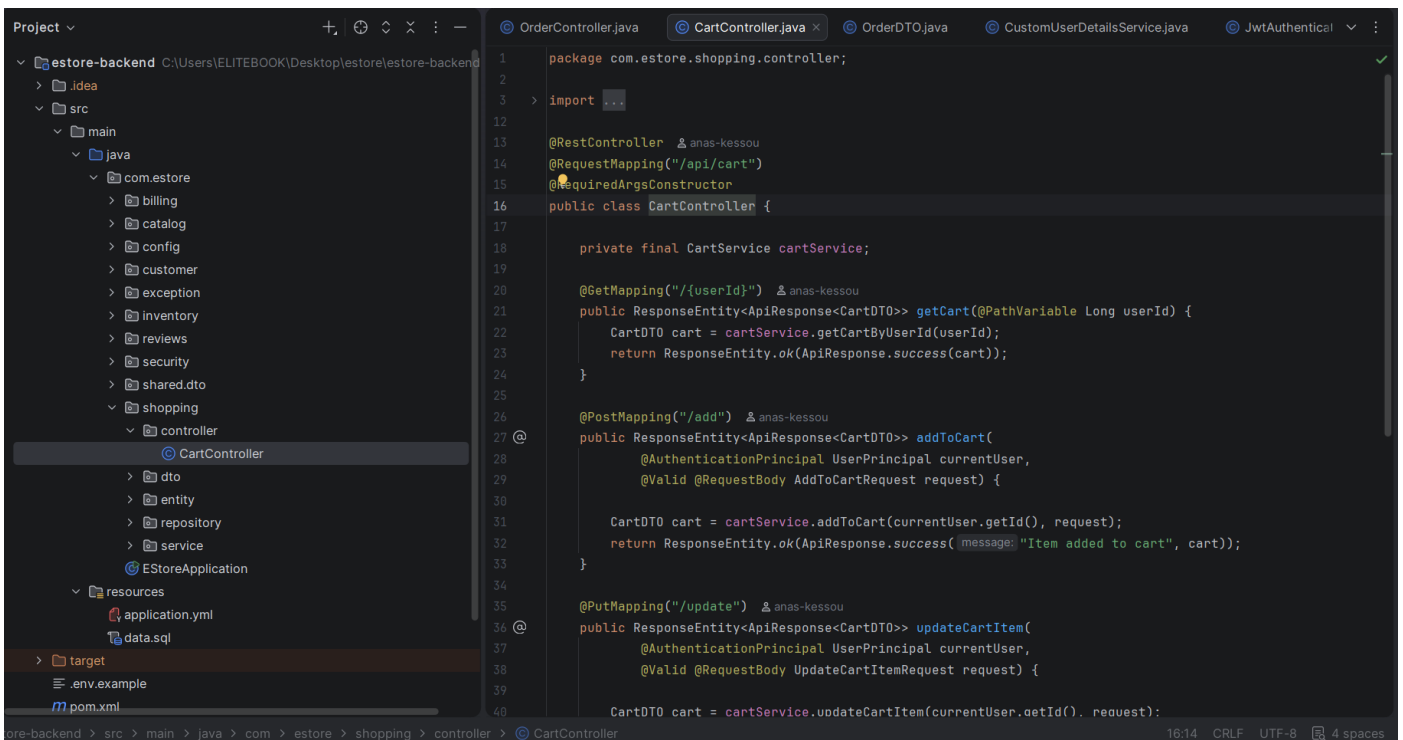
```
1 package com.estore.security;
2
3 import ...
4
5 @Getter 20 usages 2 anas-kessou
6 @AllArgsConstructor
7 public class UserPrincipal implements UserDetails {
8
9     private Long id;
10    private String email;
11    private String password;
12    private String firstName;
13    private String lastName;
14    private String role;
15    private boolean enabled;
16    private Collection<? extends GrantedAuthority> authorities;
17
18    @Override
19    public static UserPrincipal create(User user) { 2 usages 2 anas-kessou
20        return new UserPrincipal(
21            user.getId(),
22            user.getEmail(),
23            user.getPassword(),
24            user.getFirstName(),
25            user.getLastName(),
26            user.getRole(),
27            user.isEnabled(),
28            Collections.singletonList(new SimpleGrantedAuthority(user.getRole()))
29        );
30    }
31
32    @Override 2 anas-kessou
33    public String getUsername() { return email; }
```

- ProductController



```
1 package com.estore.catalog.controller;
2
3 import ...
4
5 @RestController @anas-kessou
6 @RequestMapping("/api/products")
7 @RequiredArgsConstructor
8 public class ProductController {
9
10     private final CatalogService catalogService;
11
12     @GetMapping @anas-kessou
13     public ResponseEntity<ApiResponse<PageResponse<ProductDTO>>> getProducts(
14         @RequestParam(defaultValue = "0") int page,
15         @RequestParam(defaultValue = "10") int size,
16         @RequestParam(required = false) Long categoryId) {
17
18         PageResponse<ProductDTO> products = catalogService.getProducts(page, size, categoryId);
19         return ResponseEntity.ok(ApiResponse.success(products));
20     }
21
22     @GetMapping("/{id}") @anas-kessou
23     public ResponseEntity<ApiResponse<ProductDTO>> getProductById(@PathVariable Long id) {
24         ProductDTO product = catalogService.getProductById(id);
25         return ResponseEntity.ok(ApiResponse.success(product));
26     }
27
28     @GetMapping("/search") @anas-kessou
29     public ResponseEntity<ApiResponse<PageResponse<ProductDTO>>> searchProducts(
30         @RequestParam(required = false) String q,
31         @RequestParam(required = false) Long categoryId,
32         @RequestParam(defaultValue = "0") int page,
33     ) {
34     }
35 }
```

- CartController



```
1 package com.estore.shopping.controller;
2
3 import ...
4
5 @RestController @anas-kessou
6 @RequestMapping("/api/cart")
7 @RequiredArgsConstructor
8 public class CartController {
9
10     private final CartService cartService;
11
12     @GetMapping("/{userId}") @anas-kessou
13     public ResponseEntity<ApiResponse<CartDTO>> getCart(@PathVariable Long userId) {
14         CartDTO cart = cartService.getCartByUserId(userId);
15         return ResponseEntity.ok(ApiResponse.success(cart));
16     }
17
18     @PostMapping("/add") @anas-kessou
19     public ResponseEntity<ApiResponse<CartDTO>> addToCart(
20         @AuthenticationPrincipal UserPrincipal currentUser,
21         @Valid @RequestBody AddToCartRequest request) {
22
23         CartDTO cart = cartService.addToCart(currentUser.getId(), request);
24         return ResponseEntity.ok(ApiResponse.success(message: "Item added to cart", cart));
25     }
26
27     @PutMapping("/update") @anas-kessou
28     public ResponseEntity<ApiResponse<CartDTO>> updateCartItem(
29         @AuthenticationPrincipal UserPrincipal currentUser,
30         @Valid @RequestBody UpdateCartItemRequest request) {
31
32         CartDTO cart = cartService.updateCartItem(currentUser.getId(), request);
33     }
34 }
```

- OrderController

```
1 package com.estore.billing.controller;
2
3 > import ...
4
5
6
7
8
9
10
11
12
13
14
15
16
17 @RestController @ anas-kessou
18 @RequestMapping("/api/orders")
19 @RequiredArgsConstructor
20 public class OrderController {
21
22     private final BillingService billingService;
23
24     @PostMapping @ anas-kessou
25     public ResponseEntity<ApiResponse<OrderDTO>> createOrder(
26         @AuthenticationPrincipal UserPrincipal currentUser,
27         @Valid @RequestBody CreateOrderRequest request) {
28
29         OrderDTO order = billingService.checkout(currentUser.getId(), request);
30         return ResponseEntity.ok(ApiResponse.success("Order created successfully", order));
31     }
32
33     @GetMapping("/user/{userId}") @ anas-kessou
34     public ResponseEntity<ApiResponse<PageResponse<OrderDTO>>> getUserOrders(
35         @PathVariable Long userId,
36         @RequestParam(defaultValue = "0") int page,
37         @RequestParam(defaultValue = "10") int size) {
38
39         PageResponse<OrderDTO> orders = billingService.getOrdersByUserId(userId, page, size);
40         return ResponseEntity.ok(ApiResponse.success(orders));
41     }
42
43     @GetMapping("/{orderId}") @ anas-kessou
44     public ResponseEntity<ApiResponse<OrderDTO>> getOrderById(@PathVariable Long orderId) {
```

- + Couche Service

Elle contient la logique métier du système, comme :

- Gestion des commandes

```
1 package com.estore.billing.service;
2
3 > import ...
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33 @Service @ anas-kessou
34 @RequiredArgsConstructor
35 public class BillingService {
36
37     private final OrderRepository orderRepository;
38     private final OrderItemRepository orderItemRepository;
39     private final CartRepository cartRepository;
40     private final ProductRepository productRepository;
41     private final CartService cartService;
42     private final InventoryService inventoryService;
43
44     @Transactional @Usage @ anas-kessou
45     public OrderDTO checkout(Long userId, CreateOrderRequest request) {
46         Cart cart = cartRepository.findByIdWithItems(userId)
47             .orElseThrow(() -> new ResourceNotFoundException("Cart", "userId", userId));
48
49         if (cart.getItems().isEmpty()) {
50             throw new BadRequestException("Cannot checkout with an empty cart");
51         }
52
53         // Validate stock in Inventory and prepare order items
54         for (CartItem cartItem : cart.getItems()) {
55             Long productId = cartItem.getProduct().getId();
56             int requestedQty = cartItem.getQuantity();
57
58             InventoryDTO inventory = inventoryService.getInventoryByProductId(productId);
59             if (inventory.getAvailableQuantity() < requestedQty) {
60                 throw new InsufficientStockException(productId, requestedQty, inventory.getAvailableQuantity());
```

- Vérification du stock

```
1 package com.estore.inventory.service;
2
3 > import ...
13
14 @Service & anas-kessou
15 @RequiredArgsConstructor
16 public class InventoryService {
17
18     private final InventoryRepository inventoryRepository;
19     private final ProductRepository productRepository;
20
21     @Transactional(readOnly = true) 2 usages & anas-kessou
22     public InventoryDTO getInventoryByProductId(Long productId) {
23         Inventory inventory = inventoryRepository.findById(productId)
24             .orElseThrow(() -> new ResourceNotFoundException("Inventory", "productId", productId));
25         return toInventoryDTO(inventory);
26     }
27
28     @Transactional 1 usage & anas-kessou
29     public InventoryDTO updateInventory(Long productId, InventoryUpdateRequest request) {
30         Inventory inventory = inventoryRepository.findById(productId)
31             .orElseThrow(() -> new ResourceNotFoundException("Inventory", "productId", productId));
32
33         if (request.getQuantity() != null) {
34             inventory.setQuantity(request.getQuantity());
35         }
36         if (request.getLowStockThreshold() != null) {
37             inventory.setLowStockThreshold(request.getLowStockThreshold());
38         }
39
40         inventory = inventoryRepository.save(inventory);
41     }
42 }
```

- Calcul du total

```
1 package com.estore.customer.service;
2
3 > import ...
22
23 @Service & anas-kessou
24 @RequiredArgsConstructor
25 public class CustomerService {
26
27     private final UserRepository userRepository;
28     private final ProfileRepository profileRepository;
29     private final CartRepository cartRepository;
30     private final PasswordEncoder passwordEncoder;
31     private final JwtTokenProvider tokenProvider;
32     private final AuthenticationManager authenticationManager;
33
34     @Transactional 1 usage & anas-kessou
35     @
36     public AuthResponse register(RegisterRequest request) {
37         if (userRepository.existsByEmail(request.getEmail())) {
38             throw new DuplicateResourceException("User", "email", request.getEmail());
39         }
40
41         User user = User.builder()
42             .firstName(request.getFirstName())
43             .lastName(request.getLastName())
44             .email(request.getEmail())
45             .password(passwordEncoder.encode(request.getPassword()))
46             .role("ROLE_USER")
47             .enabled(true)
48             .build();
49         user = userRepository.save(user);
50     }
51 }
```

+ Couche Repository

Elle permet l'accès aux données via Spring Data JPA et MongoDB.

```
1 package com.estore.billing.repository;
2
3 import ...
4
5 @Repository
6 public interface OrderRepository extends JpaRepository<Order, Long> {
7
8     Page<Order> findById(Long id, Pageable pageable);
9
10    Page<Order> findByUserId(Long userId, Pageable pageable);
11
12    List<Order> findByUserIdOrderByOrderDateDesc(Long userId);
13
14    Optional<Order> findByIdWithItems(@Param("orderId") Long orderId);
15
16    Page<Order> findByStatusOrderByOrderDateDesc(OrderStatus status, Pageable pageable);
17
18    @Query("SELECT COUNT(o) FROM Order o WHERE o.user.id = :userId")
19    long countByUserId(@Param("userId") Long userId);
20
21 }
```

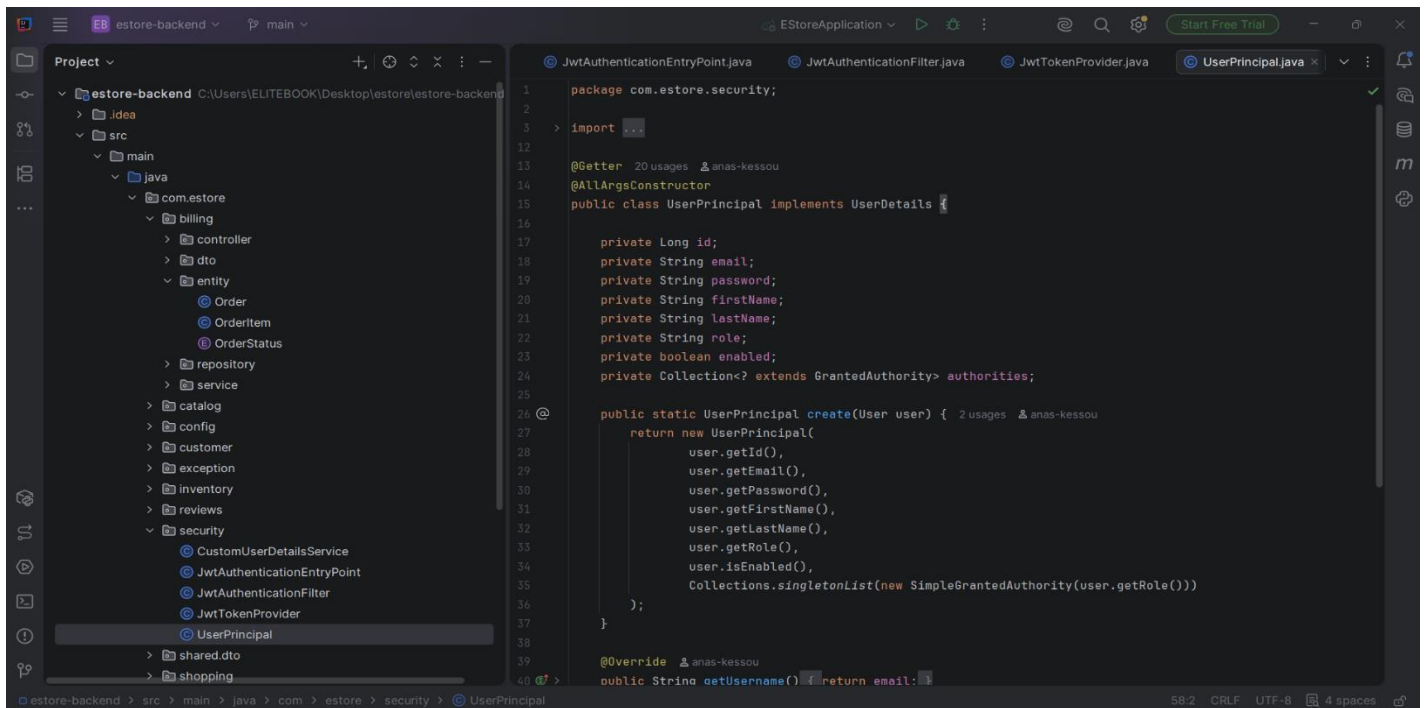
OrderRepository

ProductRepository

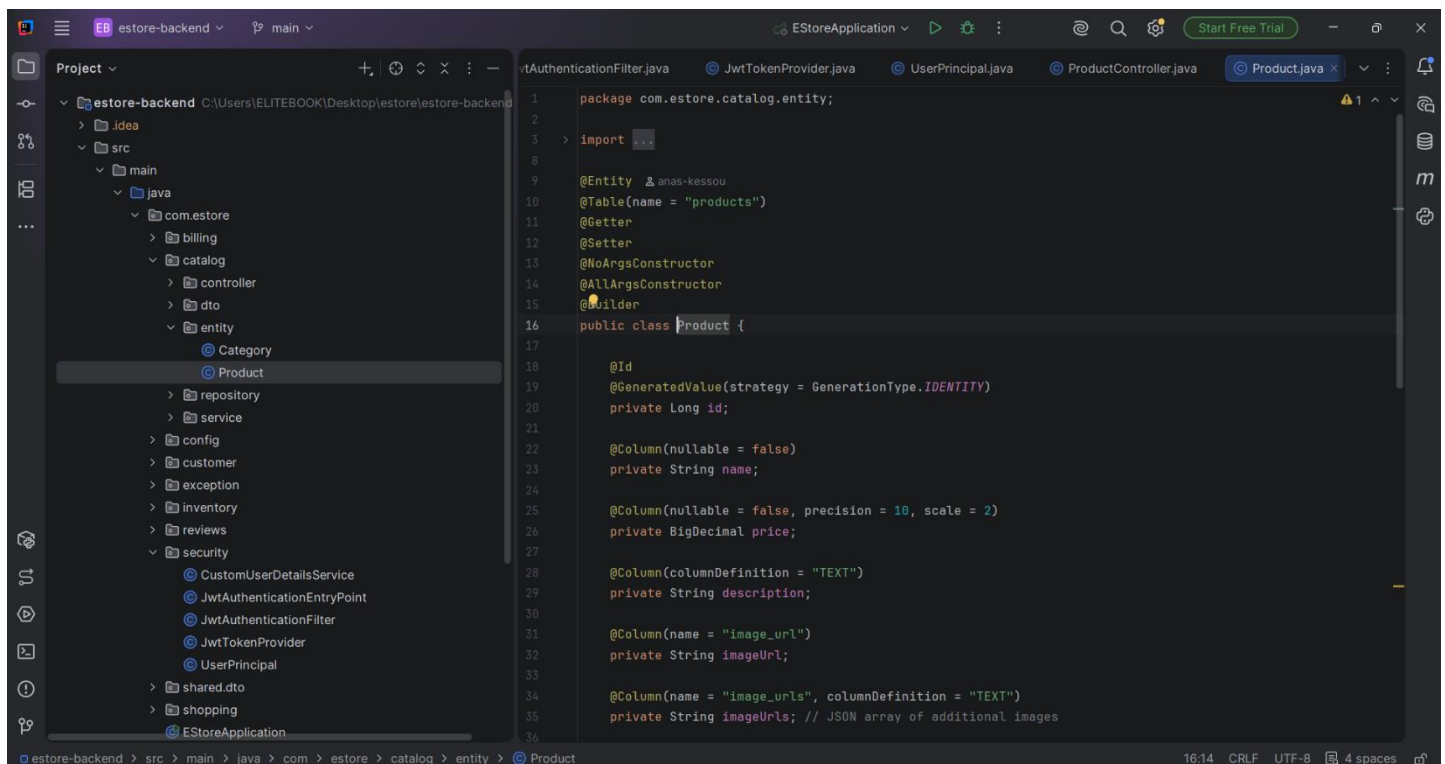
```
1 package com.estore.catalog.repository;
2
3 import ...
4
5 @Repository
6 public interface ProductRepository extends JpaRepository<Product, Long> {
7
8     Page<Product> findById(Long id, Pageable pageable);
9
10    Page<Product> findByActiveTrue(Pageable pageable);
11
12    Page<Product> findByCategoryIdAndActiveTrue(Long categoryId, Pageable pageable);
13
14    List<Product> findByFeaturedTrueAndActiveTrue(Pageable pageable);
15
16    @Query("SELECT p FROM Product p WHERE p.active = true AND " +
17            "(LOWER(p.name) LIKE LOWER(CONCAT('%', :keyword, '%')) OR " +
18            "LOWER(p.description) LIKE LOWER(CONCAT('%', :keyword, '%')))"
19    )
20    Page<Product> searchByKeyword(@Param("keyword") String keyword, Pageable pageable);
21
22    @Query("SELECT p FROM Product p WHERE p.active = true AND p.category.id = :categoryId AND " +
23            "(LOWER(p.name) LIKE LOWER(CONCAT('%', :keyword, '%')) OR " +
24            "LOWER(p.description) LIKE LOWER(CONCAT('%', :keyword, '%')))"
25    )
26    Page<Product> searchByKeywordAndCategory(@Param("keyword") String keyword,
27                                           @Param("categoryId") Long categoryId,
28                                           Pageable pageable);
29
30    List<Product> findByIdIn(List<Long> ids);
31
32    Optional<Product> findByIdAndActiveTrue(Long id);
33
34 }
```


+ Exemple de fonctionnalités réalisées

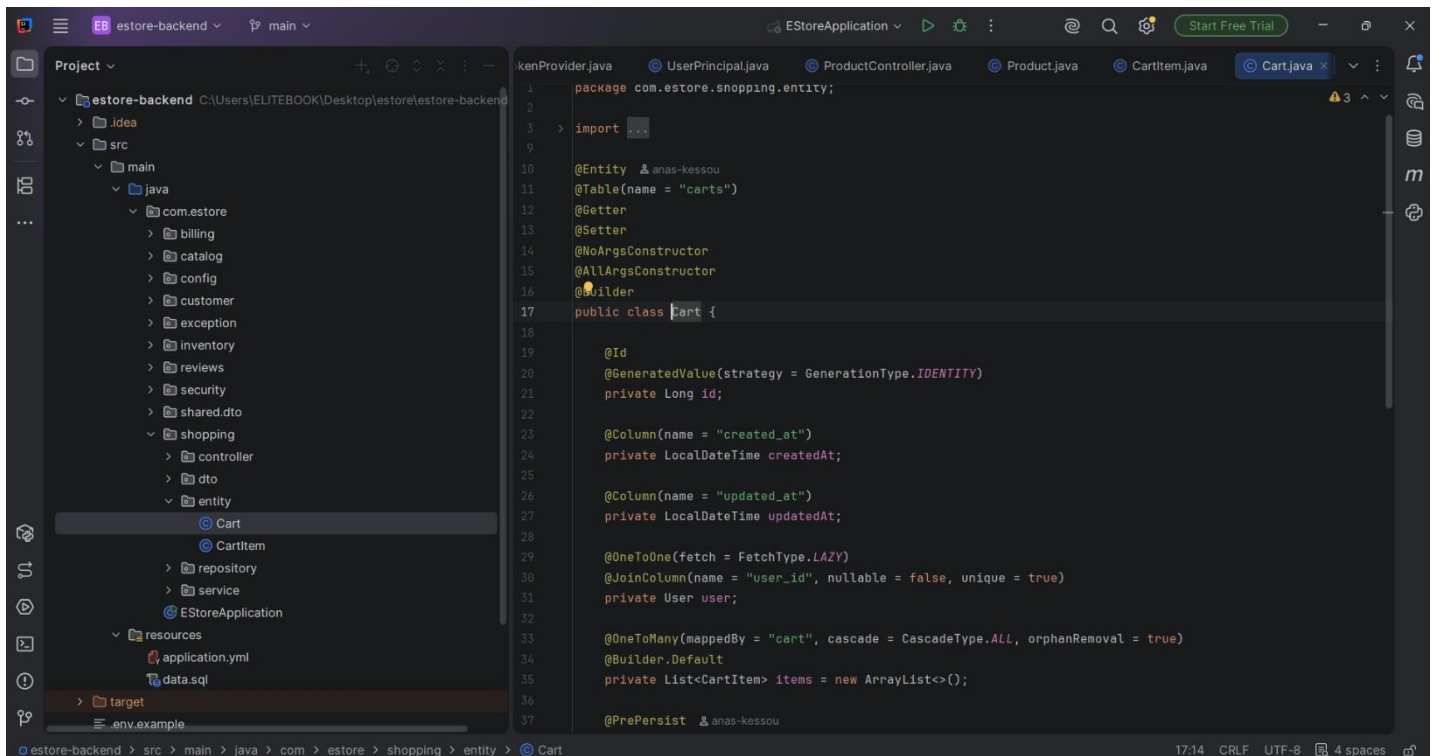
- Authentification des utilisateurs



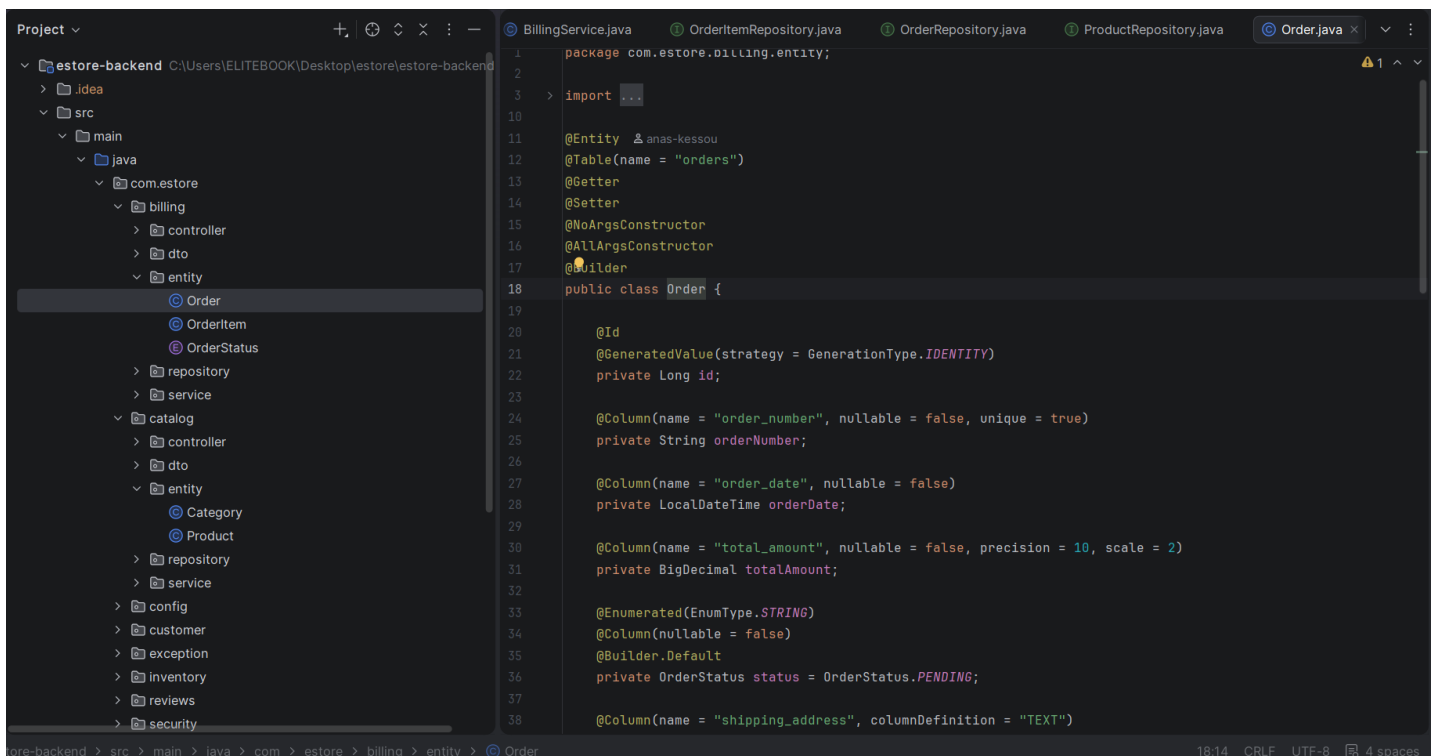
- Gestion des produits



- Gestion du panier



- Traitement des commandes



- Gestion des avis

```
1 package com.estore.reviews.document;
2
3 > import ...
4
5
6
7
8
9 @Document(collection = "reviews") 12 usages anas-kessou
10 @Getter
11 @Setter
12 @NoArgsConstructor
13 @AllArgsConstructor
14 @Builder
15 public class Review {
16
17     @Id
18     private String id;
19
20
21     @Indexed
22     private Long productId;
23
24     @Indexed
25     private Long userId;
26
27     private String authorName;
28
29     private String authorEmail;
30
31     @Indexed
32     private Integer rating;
33
34     private String comment;
35
36     @Indexed
37     private LocalDateTime createdAt;
```

5- Réalisation du Frontend

Le frontend a été développé par Angular afin de fournir une interface utilisateur dynamique et responsive.

+ Pages principales :

- Page d'accueil
- Page d'authentification (Login / Register)
- Catalogue des produits
- Détail produit
- Panier
- Page de commande

+ Fonctionnalités :

- Navigation entre les pages (Routing)
- Formulaires dynamiques
- Appels API vers le backend
- Affichage dynamique des données

6- Gestion de la base de données

Le système utilise deux types de bases de données :

+ Base relationnelle (MySQL)

Utilisée pour stocker les données structurées :

- Users
- Products
- Categories
- Orders
- Cart

+ Base NoSQL (MongoDB)

Utilisée pour les données non structurées :

- Reviews (avis des utilisateurs)

Cette approche hybride permet une meilleure flexibilité et performance dans la gestion des données.

7- Tests et validation

Afin de garantir le bon fonctionnement de l'application, plusieurs tests ont été réalisés :

✓ Tests des API

- Utilisation de Postman pour tester les endpoints
- Vérification des réponses du serveur

✓ Tests fonctionnels

- Inscription et authentification
- Ajout au panier
- Passage de commande
- Ajout d'avis

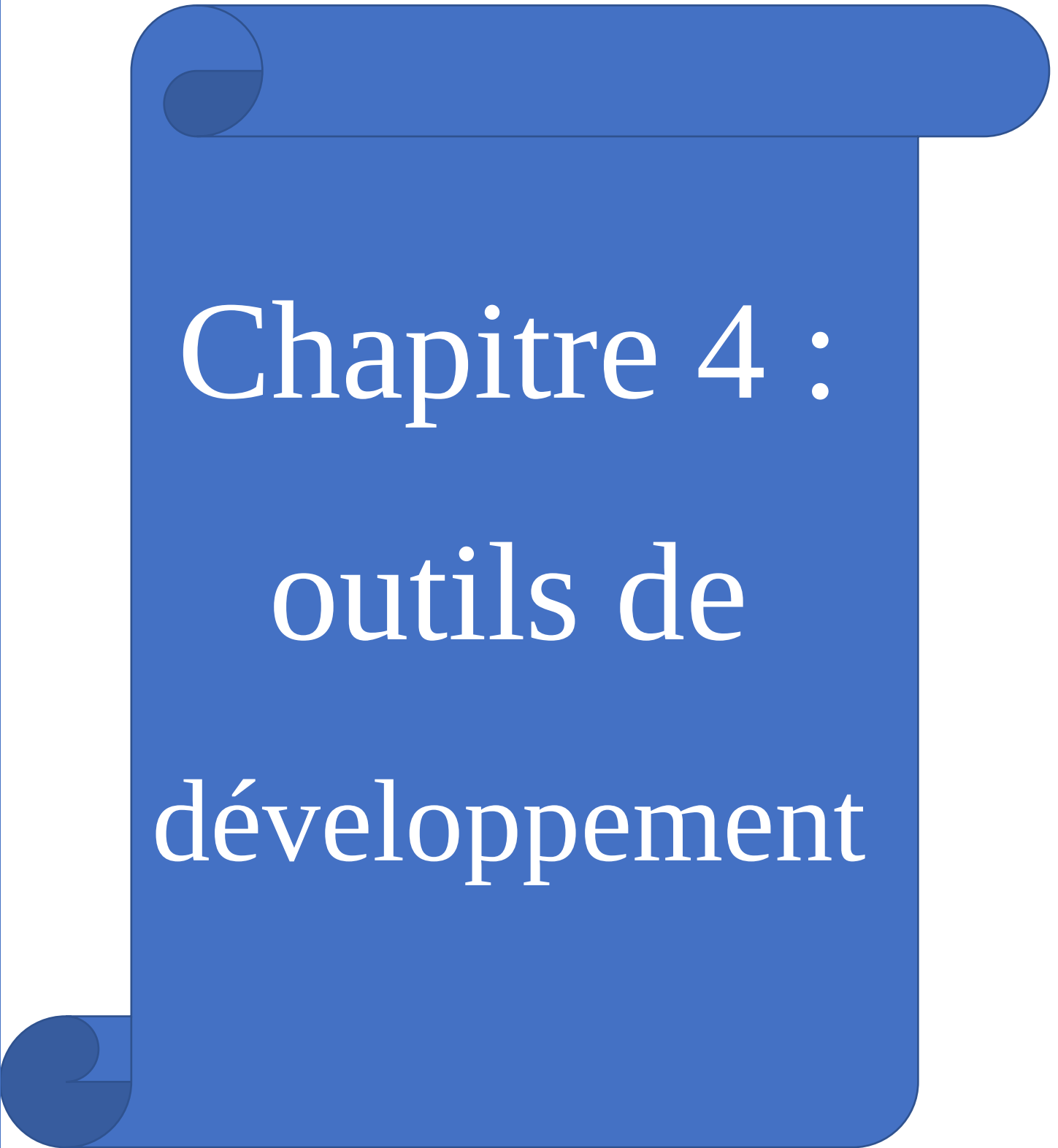
✓ Résultat

Les tests effectués ont permis de valider le bon fonctionnement global du système ainsi que la cohérence des données.

8- Conclusion

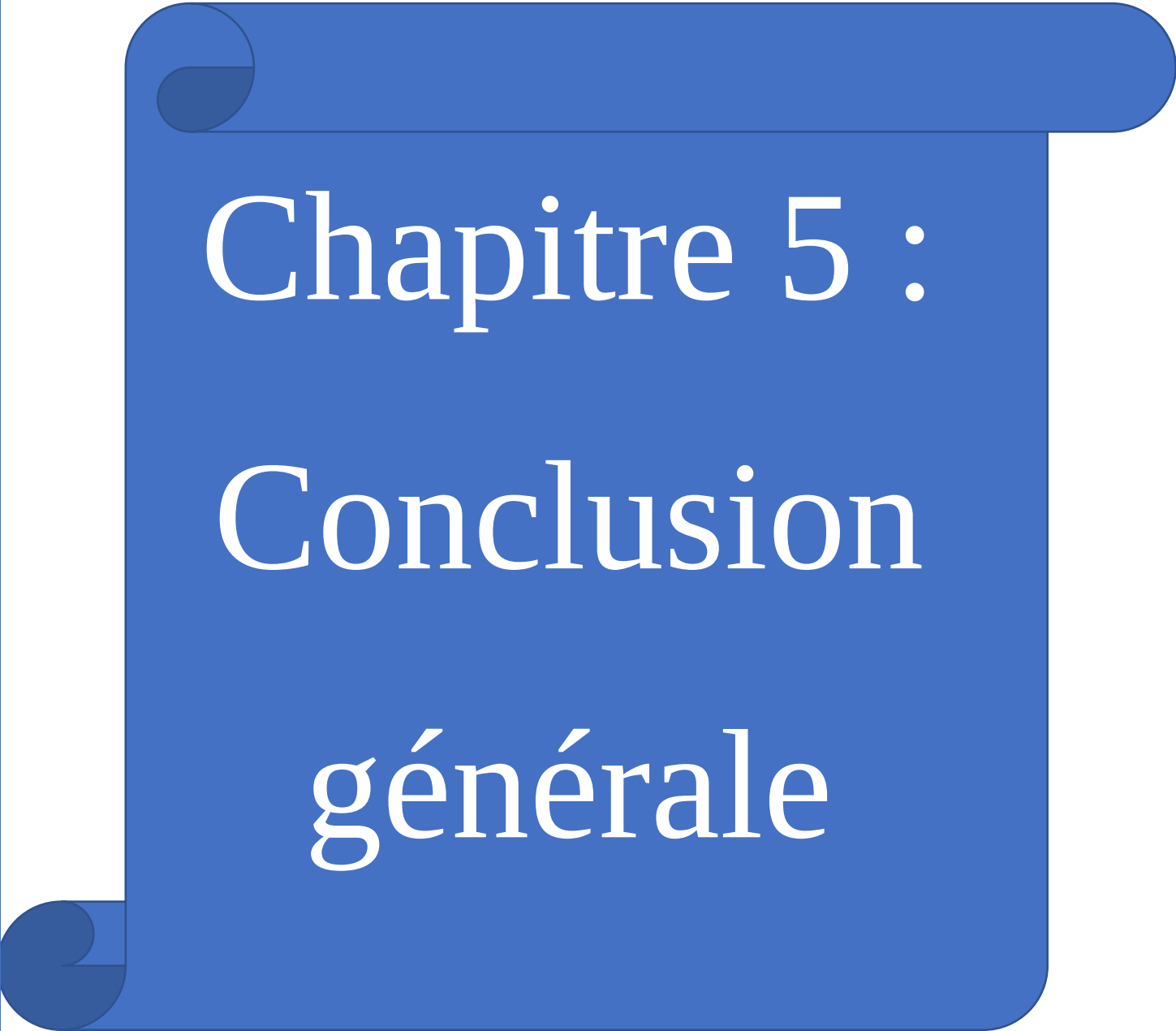
Dans ce chapitre, nous avons présenté la phase de réalisation du projet E-Store, en détaillant les outils utilisés, l'architecture adoptée ainsi que les différentes fonctionnalités implémentées.

Cette étape nous a permis de concrétiser les concepts étudiés lors de la phase de conception et de développer une application fonctionnelle répondant aux besoins définis.

A blue scroll graphic with a white border, featuring a rolled-up top edge and a rolled-up bottom edge. The text is centered on the scroll.

Chapitre 4 : outils de développement



A blue scroll graphic with a white border, featuring a rolled-up top edge and a rolled-up bottom edge. The text is centered on the scroll.

Chapitre 5 : Conclusion générale

Conclusion Générale

Le projet E-Store nous a permis de mettre en pratique les différentes connaissances acquises durant notre formation dans le domaine du développement Full Stack et de l'ingénierie logicielle. À travers la réalisation de cette application e-commerce, nous avons pu suivre les différentes étapes d'un projet informatique, depuis l'analyse des besoins jusqu'à la conception, le développement et les tests.

Ce projet nous a également permis de mieux comprendre l'importance d'une architecture logicielle bien structurée, capable d'assurer la maintenabilité, la modularité et l'évolutivité du système. L'utilisation de technologies modernes telles que Spring Boot, Angular, JPA et MongoDB nous a offert l'opportunité de développer une application dynamique, performante et adaptée aux besoins des utilisateurs.

Durant la phase de réalisation, plusieurs fonctionnalités essentielles ont été implémentées avec succès, notamment la gestion des utilisateurs, le catalogue des produits, le panier, les commandes ainsi que la gestion des avis. Les tests effectués ont permis de valider le bon fonctionnement global de l'application et la cohérence des interactions entre les différentes couches du système.

Au-delà des aspects techniques, ce projet a constitué une expérience enrichissante sur le plan méthodologique et humain. Il nous a permis de développer des compétences en travail collaboratif, en organisation, en résolution de problèmes ainsi qu'en gestion de projet.

Cependant, malgré les résultats obtenus, certaines améliorations peuvent être envisagées afin d'enrichir davantage l'application. Parmi les perspectives possibles :

- L'intégration d'un système de paiement en ligne sécurisé
- L'ajout d'un système de notifications et d'e-mails automatiques
- L'amélioration de l'interface utilisateur et de l'expérience UX
- Le déploiement de l'application sur une plateforme cloud
- L'intégration d'un système de recommandations intelligentes basé sur les préférences des utilisateurs

En conclusion, le projet E-Store représente une expérience concrète et formatrice qui nous a permis de consolider nos compétences techniques et de mieux appréhender les exigences réelles du développement d'applications web modernes.

